

Interfaz Streamlit con NLP

Módulo 4 · Código Heredado

Refactorización y Despliegue Web: NLP con Ollama + Streamlit

Ejercicio de Clase · Proyecto profesional completo con IA como copiloto

Nombre del alumno/a	
Fecha	
 Enlace GitHub	

📌 En este ejercicio partirás del código heredado `demo_nlp.py`, una demo de capacidades NLP con Ollama. Tu misión es refactorizarlo en módulos, escribir tests, configurar CI/CD, implementar almacenamiento de resultados, y sustituir la interfaz de terminal por una aplicación web con Streamlit desplegada en `interface/demo_app.py`.

BLOQUE 1 — El Código Heredado

A continuación tienes el código `demo_nlp.py`. Es funcional pero mezcla responsabilidades, no tiene interfaz de usuario, carece de tests y no persiste resultados. Tu trabajo es transformarlo en un proyecto profesional con interfaz Streamlit.

```
# demo_nlp.py - CÓDIGO HEREDADO
import os, json, sys
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(base_url='http://localhost:11434/v1', api_key='ollama')
MODELO_NLP = 'qwen2.5:0.5b'

def analizar_sentimiento(texto: str) -> dict: ...
def extraer_entidades(texto: str) -> dict: ...
def detectar_intencion(texto: str) -> dict: ...
def resumir_texto(texto: str) -> dict: ...
def clasificar_texto(texto: str) -> dict: ...

# --- Ejecución directa (mezclada con la lógica) ---
if __name__ == '__main__':
    texto = sys.argv[1] if len(sys.argv) > 1 else input('Texto: ')
    demostrar_capacidades_nlp(texto)
```

1.1 — Identifica los problemas de diseño

Sin refactorizar todavía, lista al menos 5 problemas de diseño del código (no errores de ejecución, sino malas prácticas):

#	Problema detectado	Impacto en producción

BLOQUE 2 — Modularización

 **Conexión con el módulo anterior:** tras analizar el código heredado, el primer paso profesional es separarlo en módulos con responsabilidad única (Principio SRP). El proyecto usará Streamlit para la interfaz en lugar de terminal.

2.1 — Diseña la estructura de carpetas

Define la estructura de directorios del proyecto. Indica qué archivo va en cada carpeta y por qué:

Archivo / Carpeta	Responsabilidad única	Funciones que contiene
src/cliente.py	Inicializa el cliente Ollama	get_client()
src/analizador.py		
src/niveles.py		
src/utills.py		
almacenamiento/guardar.py	Persistencia de resultados	guardar_txt(), guardar_json()
almacenamiento/leer.py		
interface/demo_app.py	Interfaz web Streamlit	app principal Streamlit
main.py	Punto de entrada CLI	Orquesta y ejecuta

2.2 — Estructura de carpetas del proyecto

```

proyecto-nlp/
├── src/
│   ├── __init__.py
│   ├── cliente.py          # Inicialización Ollama
│   ├── analizador.py      # Lógica de análisis NLP
│   ├── niveles.py         # Sentimiento / Entidades / Intención / Resumen /
├── Clasificación
│   └── utils.py           # limpiar_json(), helpers
├── almacenamiento/
│   ├── __init__.py
│   ├── guardar.py         # Guardar TXT y JSON
│   └── leer.py            # Leer histórico
├── resultados/
│   ├── txt/              # Un .txt por análisis
│   └── json/             # Un .json por análisis
├── interface/
│   ├── __init__.py
│   └── demo_app.py        # App Streamlit (interfaz web)
├── tests/
│   ├── test_analizador.py
│   └── test_almacenamiento.py
├── docs/
├── heredado/
│   └── demo_nlp.py
├── .env
├── .gitignore
├── requirements.txt
└── README.md
    
```

2.3 — ¿Por qué Streamlit y no Tkinter?

El proyecto original usaba Tkinter. En este ejercicio la interfaz se implementa con Streamlit. Completa la tabla comparativa:

Criterio	Tkinter	Streamlit
Instalación	Incluida en Python estándar	pip install streamlit
Paradigma	Programación orientada a eventos	Scripts Python ejecutados de arriba a abajo
Despliegue web		
Curva de aprendizaje		
Actualización UI		
Uso en empresas		

2.4 — Prompt para que la IA refactorice

Redacta el prompt completo que le darías a la IA. Incluye los 5 elementos del prompt profesional:

2.5 — Escribe el código refactorizado

Pega aquí el código de cada módulo tras la refactorización:

```
# src/analizador.py
# (escribe aquí tu código)
```

```
# src/niveles.py
# (escribe aquí tu código)
```

```
# almacenamiento/guardar.py
# (escribe aquí tu código)
```

BLOQUE 3 — Tests Unitarios

 **Regla profesional:** nunca se sube código a producción sin tests. En este bloque escribirás tests para cada módulo usando pytest, incluyendo tests para el módulo de almacenamiento (TXT y JSON) y para la lógica NLP.

3.1 — Prompt para que la IA genere los tests

Escribe el prompt que usarías para que la IA genere los tests unitarios. Recuerda pedir: casos felices, casos borde y casos de error:

3.2 — Escribe al menos 6 casos de test

Crea el archivo tests/test_analizador.py con mínimo 6 casos de prueba. Planifícalos antes de escribir el código:

Nombre del test	Tipo (feliz/borde/error)	Input / condición	Resultado esperado
test_1_			
test_2_			
test_3_			
test_4_			
test_5_			
test_6_almacenamiento_txt	feliz	Resultado válido	Archivo .txt creado en resultados/txt/
test_7_almacenamiento_json	feliz	Resultado válido	Archivo .json creado en resultados/json/

3.3 — Código de los tests

```
# tests/test_analizador.py
import pytest
from src.analizador import analizar_sentimiento
from almacenamiento.guardar import guardar_resultado
import os, json

def test_sentimiento_positivo():
    # ...
    pass

def test_guardar_txt_crea_archivo():
    # Verifica que el archivo TXT se crea en resultados/txt/
    pass

def test_guardar_json_crea_archivo():
    # Verifica que el archivo JSON se crea en resultados/json/
    pass
```

BLOQUE 4 — Pipeline CI/CD con GitHub Actions

 **Conexión con el módulo de Pipeline:** con el código modularizado y los tests escritos, el último paso es automatizar la validación. Cada vez que hagas git push, GitHub Actions ejecutará linter, tests y análisis de seguridad automáticamente.

4.1 — Diseña el fichero .github/workflows/ci.yml

Indica qué debe hacer cada fase del pipeline y qué herramienta usarías:

Fase del pipeline	¿Qué debe hacer?	Comando / herramienta
1. Checkout del código		actions/checkout@v4
2. Configurar Python		actions/setup-python@v5
3. Instalar dependencias		pip install -r requirements.txt
4. Ejecutar linter		flake8 / ruff
5. Tests unitarios		pytest
6. Cobertura de tests		pytest --cov
7. Verificar carpetas		python scripts/check_folders.py
8. Análisis de seguridad		bandit

4.2 — Escribe el fichero ci.yml completo

```
# .github/workflows/ci.yml
name: CI Pipeline - NLP Ollama + Streamlit

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      # Completa aquí cada step del pipeline
```

4.3 — ¿Qué ocurre si un test falla en el pipeline?

Explica el flujo: ¿se bloquea el merge? ¿Quién recibe la notificación? ¿Cómo se depura el error?

BLOQUE 5 — Almacenamiento de Resultados (TXT y JSON)

 *Requisito profesional: los resultados de entrada y salida deben persistir automáticamente en carpetas separadas. Carpeta resultados/txt/ para lectura humana y resultados/json/ para integración con sistemas externos.*

5.1 — Diseña el módulo almacenamiento/guardar.py

El módulo debe crear las carpetas si no existen, guardar con timestamp y no sobrescribir archivos anteriores:

```
# almacenamiento/guardar.py
import os, json
from datetime import datetime
from pathlib import Path

CARPETA_TXT = Path('resultados/txt')
CARPETA_JSON = Path('resultados/json')

def _crear_carpetas():
    CARPETA_TXT.mkdir(parents=True, exist_ok=True)
    CARPETA_JSON.mkdir(parents=True, exist_ok=True)

def guardar_resultado(texto_entrada: str, resultados: dict) -> dict:
    """Guarda en TXT y JSON. Devuelve las rutas de los archivos creados."""
    _crear_carpetas()
    timestamp = datetime.now().strftime('%Y-%m-%d_%H%M%S')
    nombre_base = f'análisis_{timestamp}'
    # (escribe aquí tu código para TXT y JSON)
```

```
return { 'txt': str(ruta_txt), 'json': str(ruta_json) }
```

5.2 — Diseña el módulo almacenamiento/leer.py

Función	Descripción	Retorna
listar_analisis()	Lista todos los archivos guardados	list[str]
leer_json(nombre)	Lee un análisis en formato JSON	dict
leer_txt(nombre)	Lee un análisis en formato TXT	str
buscar_por_fecha(fecha)	Filtra por fecha	list[str]

5.3 — Formato de los archivos de salida

```
# Formato de nombre de archivo:
# resultados/txt/ analisis_2025-04-08_143022.txt
# resultados/json/ analisis_2025-04-08_143022.json

# Contenido TXT (entrada + salida legible):
# =====
# ANÁLISIS NLP — 2025-04-08 14:30:22
# =====
# TEXTO ANALIZADO:
# El producto llegó rápido, pero la calidad no es lo que esperaba.
#
# SENTIMIENTO: negativo | puntuación: -0.4 | confianza: 0.87
# INTENCIÓN: queja | urgencia: alta
# CLASIFICACIÓN: soporte | prioridad: 4/5

# Contenido JSON:
# { 'timestamp': '...', 'texto': '...', 'sentimiento': {...},
#   'entidades': {...}, 'intencion': {...}, 'resumen': {...} }
```

BLOQUE 6 — Documentación Profesional

 *Un proyecto sin documentación es un proyecto muerto. En este bloque crearás el README y la carpeta docs/ siguiendo estándares profesionales. Incluye la documentación del sistema de almacenamiento y las instrucciones de Streamlit.*

6.1 — Estructura de la carpeta docs/

Archivo en docs/	Contenido / propósito
arquitectura.md	Diagrama de módulos y flujo de datos
almacenamiento.md	Documentación del sistema TXT y JSON
api_referencia.md	Referencia de funciones públicas
streamlit_guia.md	Cómo ejecutar y configurar la interfaz Streamlit

6.2 — Escribe el README.md del proyecto

El README debe incluir: descripción, instalación (incluyendo Streamlit), uso, estructura de carpetas, cómo ejecutar los tests y badge del pipeline CI:

```
# README.md
# NLP con Ollama + Streamlit

## Descripción

## Instalación
```bash
pip install streamlit openai python-dotenv
```

## Uso — Interfaz web
```bash
streamlit run interface/demo_app.py
```

## Estructura del proyecto
```
```

## Almacenamiento de resultados
# Explica aquí cómo se guardan los TXT y JSON

## Tests
```bash
pytest tests/
```

## Pipeline CI

## Licencia
```

6.3 — Commits semánticos

Indica qué mensaje de commit escribirías para cada cambio. Usa el formato: tipo(scope): descripción corta

| Tipo de cambio | Tu mensaje de commit | ¿Por qué este mensaje? |
|----------------|----------------------|------------------------|
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |



Interfaz Streamlit con NLP



| | | |
|--|--|--|
| | | |
|--|--|--|

BLOQUE 7 — Resultado Final — Interfaz Streamlit

El resultado final del proyecto debe incluir la interfaz web desarrollada con Streamlit (en `interface/demo_app.py`), que muestra los 5 análisis NLP en tiempo real y confirma el guardado automático de resultados.

La aplicación se arranca con un solo comando desde la raíz del proyecto:

```
streamlit run interface/demo_app.py
```

La interfaz debe mostrar:

| Componente de la interfaz | Descripción | Obligatorio |
|----------------------------|--|-------------|
| Sidebar con ejemplos | 3 textos de ejemplo cargables con un clic | Sí |
| Área de texto de entrada | Campo para introducir el texto a analizar | Sí |
| Botón Analizar | Lanza los 5 análisis NLP | Sí |
| Tab Sentimiento | Badge de color + barras de puntuación y confianza | Sí |
| Tab Entidades (NER) | Grid de entidades por categoría con iconos | Sí |
| Tab Intención | Intención, subcategoría, urgencia, acción sugerida | Sí |
| Tab Resumen | 3 expanders: ultracorto, medio, detallado | Sí |
| Tab Clasificación | Tema, tipo, canal y barra de prioridad | Sí |
| Barra de estado / guardado | Confirma guardado automático con ruta del archivo | Sí |
| Historial (opcional) | Lista de análisis previos guardados | No |

 El guardado automático debe activarse tras cada análisis exitoso. La barra de estado debe mostrar: '✓ Análisis completado y guardado automáticamente' con la ruta del archivo generado.

7.1 — Diseña el flujo de la aplicación Streamlit

Describe con tus propias palabras el flujo de ejecución de la app cuando el usuario pulsa el botón Analizar. ¿Qué funciones se llaman? ¿En qué orden? ¿Qué se muestra?

| |
|--|
| |
| |
| |
| |
| |

7.2 — Diferencias entre ejecución CLI y Streamlit

| Aspecto | Ejecución CLI (demo_nlp.py) | Interfaz Streamlit (demo_app.py) |
|----------------------|-----------------------------|----------------------------------|
| Entrada de texto | sys.argv o input() | st.text_area() |
| Salida de resultados | print() en terminal | st.tabs() + st.markdown() |
| Re-ejecución | Nuevo comando cada vez | |
| Estado de la app | Efímero (termina al acabar) | |
| Guardado | | |

BLOQUE 8 — Reflexión Integrada

 **Reflexión final:** Describe el flujo profesional completo que acabas de implementar. ¿Qué aporta cada paso (análisis → modularización → tests → CI/CD → almacenamiento → interfaz Streamlit → docs)? ¿Qué hubiera pasado si te saltas alguno?

8.1 — Flujo profesional completo

Describe con tus palabras qué aprendiste en cada fase y por qué el orden importa:

| |
|--|
| |
| |
| |
| |
| |
| |

8.2 — ¿Cómo usaste la IA como copiloto?

Para cada bloque, indica en qué momento pediste ayuda a la IA y cómo validaste su respuesta antes de aplicarla:

| Bloque | ¿Para qué usé la IA? | ¿Cómo verifiqué su respuesta? |
|--------|----------------------|-------------------------------|
| | | |
| | | |
| | | |
| | | |
| | | |
| | | |

8.3 — Autoevaluación

Marca con una X tu nivel de confianza al finalizar el ejercicio:

| Nivel | Descripción | ¿Qué necesitarías repasar? |
|----------------------------|-------------------------|----------------------------|
| ☐ 1 | Necesito más práctica | |
| ☐ 2 | Lo entiendo con ayuda | |
| ☐ 3 | Lo aplico con seguridad | |
| ☐ 4 | Podría enseñarlo | |

BLOQUE 9 — Reflexión MemoriaFinal.md

En la carpeta reflexiones/, crea un archivo llamado memoriaFinal.md donde explicaréis el porqué el código original está mal configurado y qué cambios realizasteis. Incluye también la decisión de usar Streamlit en lugar de Tkinter.

9.1 — Estructura sugerida del memoriaFinal.md

```
# Memoria Final — Refactorización NLP con Streamlit

## 1. Problemas detectados en el código heredado
- ...

## 2. Decisiones de diseño tomadas
- Separación en módulos (SRP)
- Elección de Streamlit sobre Tkinter: ...

## 3. Cambios realizados por bloque
- Bloque 1 — Análisis: ...
- Bloque 2 — Modularización: ...
- Bloque 3 — Tests: ...
- Bloque 4 — CI/CD: ...
- Bloque 5 — Almacenamiento: ...
- Bloque 6 — Docs: ...
- Bloque 7 — Streamlit: ...

## 4. Lecciones aprendidas
- ...

## 5. ¿Qué mejorarías con más tiempo?
- ...
```

9.2 — Escribe aquí tu reflexión (borrador)

| |
|--|
| |
| |
| |
| |
| |
| |